AdaThink: Adaptive Budget Allocation for Test-Time Compute in Large Language Models

Anonymous Author(s)

Abstract

Scaling test-time compute by generating longer chain-of-thought reasoning has become a dominant strategy for improving LLM accuracy on hard problems. However, we observe that *overthinking*—spending excess reasoning tokens on questions that do not benefit from extended deliberation—wastes compute without improving accuracy, and sometimes actively hurts it. We introduce AdaThink, a training-free framework that allocates reasoning tokens per question based on predicted difficulty, requiring no model modification or fine-tuning. Our primary contribution is a *template-based controller*—a lookup table over three binary difficulty features—that closes 76% of the gap to an oracle budget selector while remaining trivially reproducible. The controller is optimized via leave-one-subset-out cross-validation with an explicit cost-penalizing utility objective. Evaluated across three benchmarks (GSM8K, MATH500, BIG-Bench Hard) and two model scales (Qwen3-8B and Qwen3.5-27B), the template controller achieves **+11.3–28.9** percentage-point accuracy gains over the matched fixed-budget baseline. Despite the overhead of a two-pass procedure (probe at low budget, then full generation at the selected budget), all five template-controller settings show positive utility gains (+7.4–23.1 pp) because accuracy improvements outweigh the additional cost. All accuracy gains are statistically significant under paired bootstrap confidence intervals. Ablation studies show that access to the highest budget tier is critical (removing it causes severe accuracy drops on smaller models), while the optimal number of tiers is setting-dependent.

1 Introduction

Large language models (LLMs) increasingly rely on extended chain-of-thought (CoT) reasoning at inference time to solve complex tasks [Wei et al., 2022, OpenAI, 2024]. The prevailing paradigm is straightforward: allocate more tokens, produce longer reasoning traces, and achieve higher accuracy. Recent work on test-time compute scaling [Snell et al., 2024] has formalized this intuition, showing that accuracy often improves log-linearly with the number of generated tokens.

However, this "more is better" assumption breaks down in practice. We identify a pervasive *overthinking* phenomenon: for a substantial fraction of questions, extending the reasoning budget beyond a certain threshold yields no accuracy gain—and sometimes causes regressions as the model "talks itself out of" correct intermediate answers. On GSM8K with Qwen3.5-27B, increasing the budget from 256 to 512 tokens costs 2× more compute but yields only $\Delta\text{Acc} = +2.5\,\text{pp}$—a marginal gain that does not justify doubling the cost. Conversely, many hard questions genuinely require extended reasoning and are severely harmed by premature truncation.

39th Conference on Neural Information Processing Systems (NeurIPS 2026).

This observation motivates a natural question: *can we dynamically allocate reasoning tokens per question, spending more on hard problems and less on easy ones, to improve the quality–cost Pareto frontier?*

We introduce AdaThink, a framework for adaptive test-time compute allocation. Rather than applying a uniform budget across all inputs, AdaThink uses a learned controller to predict the appropriate reasoning budget for each question based on difficulty features extracted from partial decoding traces. Our framework encompasses three controller variants of increasing sophistication:

1. A **template controller** that maps discrete difficulty features to budget tiers via a lookup table optimized through leave-one-subset-out cross-validation.
2. A **parametric controller** that uses a linear policy over continuous features with a constrained optimization objective.
3. A **value-based controller** that predicts per-budget correctness probabilities and selects budgets via a cost-aware decision rule.

All controllers optimize a unified utility objective $U = \text{Accuracy} - \lambda \cdot \text{NormalizedCost}$ that explicitly trades off quality against compute, with λ controlling the cost sensitivity. Training uses per-sample decoding traces from multi-seed replicated experiments, and evaluation follows a strict leave-one-subset-out protocol to prevent data leakage.

We evaluate AdaThink on three diverse benchmarks—GSM8K (mathematical reasoning), MATH500 (competition mathematics), and BIG-Bench Hard (mixed reasoning)—using two model scales (Qwen3-8B and Qwen3.5-27B). Our key findings are:

- **Large and statistically significant gains.** The template controller achieves +14.2 pp absolute accuracy on GSM8K-27B, +12.1 pp on MATH500-27B, and +11.3 pp on BBH-27B over the matched fixed-budget baseline, with all 95% bootstrap CIs excluding zero.
- **Cross-scale generalization.** Gains persist on the smaller 8B model: +28.9 pp on MATH500-8B and +12.1 pp on BBH-8B, confirming that adaptive control is not limited to large models.
- **Positive utility despite two-pass overhead.** The two-pass procedure (probe at b_1 followed by generation at the selected budget) adds token overhead, but utility (accuracy minus normalized cost) improves across all five template-controller settings (+7.4 to +23.1 pp) because accuracy gains consistently outweigh the additional cost.
- **Component contributions validated.** Ablation studies across five settings show that access to the highest budget tier is critical (removing it causes severe accuracy drops on 8B models), while the value of intermediate tiers is setting-dependent.
- **Stronger baselines.** We compare against fair self-consistency variants (SC@2×256, SC@4×128) and a continuation-from-b_1 baseline. All are substantially outperformed by the template controller, and multi-seed analysis confirms zero stochastic decoding variance under greedy generation.

Our central scientific finding is that *fixed test-time compute systematically overthinks easy questions and undercomputes hard ones*, and this heterogeneity is so structured that a tiny binary-feature controller can exploit it. A lookup table over three binary features—requiring no training, no model modification, and $\leq$ 6561 exhaustive evaluations—closes 76% of the gap to an oracle budget selector on GSM8K-27B. Feature-stratified analysis reveals the mechanism: ∼13% of questions are "easy" (correct at the lowest budget with 60% overthinking rate at the highest), while ∼60% are "hard" (zero accuracy at the lowest budget); routing each stratum to its optimal tier yields large gains with lower compute than any fixed budget. This establishes a strong, reproducible lower bound on what post-hoc budget control can achieve *without any model modification* for the growing literature on adaptive test-time compute [Liu et al., 2025, Yang et al., 2025, Han et al., 2025, Li et al., 2025]. We validate across mathematical and logical reasoning with two Qwen model scales; cross-family generalization is future work.

2 Related Work

Test-time compute scaling. Recent work has demonstrated that allocating additional compute at inference time—through longer generation, multiple samples, or iterative refinement—can substantially improve LLM performance [Snell et al., 2024, OpenAI, 2024, Muennighoff et al., 2025a]. Snell et al. [2024] show log-linear accuracy scaling with compute budget on mathematical reasoning. Models like o1 [OpenAI, 2024] and DeepSeek-R1 [DeepSeek-AI, 2025] are specifically trained for extended reasoning but apply uniform budgets, motivating per-instance adaptation.

Adaptive reasoning budgets. Our work is most directly related to a growing body of methods that adapt per-instance reasoning effort. **Training-time approaches** modify the model to produce difficulty-calibrated traces. TALE [Han et al., 2025] includes a token budget in prompts and trains via SFT/DPO to compress reasoning with minimal accuracy loss. SelfBudgeter [Liu et al., 2025] predicts the required budget *before* generation and enforces adherence via budget-guided GRPO, achieving 61% average length compression. BudgetThinker [Yang et al., 2025] inserts periodic control tokens to inform the model of remaining budget during inference, trained with curriculum RL. DRER [Zhu et al., 2025] couples reward attribution to token-level reasoning quality with a dynamic length advantage. **Inference-time approaches** leave the base model unchanged. Plan-and-Budget [Li et al., 2025] decomposes queries into sub-questions and allocates tokens via decay-based heuristics. Parallel-Probe [Zheng et al., 2025] jointly optimizes width and depth in parallel reasoning via consensus-based early stopping and branch pruning. Aggarwal et al. [2023] explore heuristic stopping rules for adaptive token budgets.

AdaThink is an inference-time, training-free controller that differs from the above in two key ways: (i) it requires no model modification or fine-tuning, operating purely on cached decoding traces; (ii) it optimizes an explicit cost-aware utility via exhaustive search over a compact template space. Compared to SelfBudgeter (which trains the model for budget prediction), AdaThink trades training cost for zero deployment friction. Compared to Plan-and-Budget (which uses handcrafted decay heuristics), AdaThink learns the allocation from data. The approach is complementary to training-time methods: a SelfBudgeter-tuned model could use an AdaThink controller as a post-hoc refinement layer.

Self-consistency and verification. Self-consistency [Wang et al., 2023] improves accuracy via majority voting over multiple samples, uniformly multiplying compute. Verifier-based approaches [Cobbe et al., 2021, Lightman et al., 2024] train separate models to score candidates. AdaThink *reallocates* compute across questions rather than uniformly increasing it, and can use consistency features as controller inputs.

Complementary runtime controllers. Cascade-style speculative decoding [Chen et al., 2023] selects speculation length per step; KV-cache eviction controllers [Zhang et al., 2024] manage memory budgets. Both address different resources (latency, memory) but share the philosophy of lightweight, learned reallocation; AdaThink targets per-question CoT budget and could compose with these for joint optimization.

3 Method

3.1 Problem Formulation

Consider an LLM $\mathcal{M}$ that generates a reasoning trace y for a question x under a token budget $b \in \mathcal{B} = \{b_1, b_2, \ldots, b_K\}$, where $b_1 < b_2 < \cdots < b_K$. Let $c(x, b) \in \{0, 1\}$ denote the correctness of the answer extracted from the trace generated under budget b, and let $t(x, b)$ denote the actual number of tokens consumed.

A *fixed-budget* policy $\pi_{\text{fix}}(x) = b_k$ assigns the same budget to all questions. We seek an *adaptive* policy $\pi : \mathcal{X} \to \mathcal{B}$ that selects a per-question budget to maximize a cost-aware

utility:

$$U(\pi) = \mathbb{E}_x \left[c\big(x, \pi(x)\big) - \lambda \cdot \frac{t\big(x, \pi(x)\big)}{b_K} \right], \tag{1}$$

where $\lambda \geq 0$ controls the cost penalty and b_K normalizes token counts so that a single full-budget pass has cost ≈ 1.[1] We seek π^* that improves the quality–cost trade-off relative to uniform fixed-budget decoding, as governed by λ.

3.2 Template Budget Controller

Our primary controller uses a template-based approach that maps *difficulty features* to budget tiers.

Difficulty features. From a partial decoding trace at the lowest budget b_1, we extract:

- **Answer presence**: whether a parseable final answer appears within b_1 tokens.
- **Token utilization**: the fraction $t(x, b_1)/b_1$, binarized at threshold 0.95 ("high" if ≥ 0.95, "low" otherwise).
- **Answer consistency**: for models with thinking mode, whether the intermediate reasoning and final answer agree (binary).

All three features are binary, yielding at most $D = 2^3 = 8$ difficulty categories $d(x) \in \{1, \ldots, D\}$. Answer presence is determined by regex-based parsing: for GSM8K, we search for "Final answer: ⟨number⟩" or extract the last number; for MATH500, we extract `\boxed{}` contents; for BBH, we match option letters or the "The answer is" pattern. When strict parsing fails and the budget is exhausted without a parseable answer, we optionally extend generation by up to 16 tokens ("projection"; Appendix A).

We chose these three features because they are (i) available from a single low-budget pass without additional model calls, (ii) interpretable as difficulty proxies (questions answered quickly and consistently are likely easier), and (iii) sufficient to produce a compact partition ($D \leq 8$) that avoids overfitting. Richer features such as token-level logprob trends or lightweight verifier scores could improve discrimination but would require additional inference-time computation; we leave this exploration to future work (Section 5).

Budget assignment. The template controller maintains a mapping $T : \{1, \ldots, D\} \to \mathcal{B}$ that assigns each difficulty category to a budget tier. The policy is:

$$\pi_{\text{template}}(x) = T\big(d(x)\big). \tag{2}$$

Training via leave-one-subset-out cross-validation. Given N per-sample traces from replicated experiments (each drawing n_s questions uniformly at random from the benchmark), we optimize T by leave-one-subset-out cross-validation:

1. For each held-out subset S_i, compute the utility (1) of every possible template assignment on the remaining $N - |S_i|$ samples.
2. Select the template T_i^* that maximizes training utility.
3. Evaluate T_i^* on the held-out S_i to obtain an unbiased performance estimate.

We report performance by pooling all held-out evaluations. Because subsets are independent random draws, a small fraction of questions may appear in multiple subsets (e.g., $\sim 27\%$ of sample-appearances on GSM8K-27B). This does not compromise evaluation validity: the controller maps abstract difficulty features to budget tiers and cannot exploit question identity, and the held-out fold is never used during template selection for that fold.[2]

Algorithm 1 summarizes the AdaThink inference pipeline.

[1] Actual token counts $t(x, b)$ may slightly exceed b due to answer-projection tokens; the normalization by b_K keeps the cost term on an interpretable scale, though it can slightly exceed 1 for the maximum tier and exceeds 1 for two-pass questions.

[2] We verified empirically that pairwise subset overlap is small (mean 1.6 out of 40 questions); removing overlapping questions from the training fold yields statistically indistinguishable results.

Algorithm 1 AdaThink Inference

Require: Question x, LLM $\mathcal{M}$, budget tiers $\mathcal{B} = \{b_1, \ldots, b_K\}$, controller π
Ensure: Answer a, tokens consumed t

```
Generate trace y_1 ← M(x, b_1)                          {Initial low-budget pass}
Extract features d(x) ← Features(x, y_1)
Select budget b* ← π(d(x))                              {Controller decision}
if b* = b_1 then
    a ← ParseAnswer(y_1); t ← t(x, b_1)
else
    Generate trace y* ← M(x, b*)                        {Fresh full-budget pass}
    a ← ParseAnswer(y*); t ← t(x, b_1) + t(x, b*)       {End-to-end cost}
end if
return a, t
```

3.3 Value-Based Controller

For finer-grained cost control, we introduce a value-based controller that predicts *per-budget correctness probabilities*.

Value function. For each budget b_k, we estimate $\hat{V}(x, b_k) = \hat{P}(c(x, b_k) = 1)$ using the empirical correctness rate within the difficulty category $d(x)$, computed from training traces. Specifically, $\hat{V}(x, b_k) = \frac{1}{|C_{d(x)}|} \sum_{x' \in C_{d(x)}} c(x', b_k)$, where $C_{d(x)}$ is the set of training questions sharing the same difficulty category. No additional smoothing is applied; categories with fewer than 5 training samples fall back to the corpus-wide mean to mitigate sparsity (see Appendix B.12 for a calibration analysis).

Cost-aware decision rule. Given a penalty parameter $\rho \geq 0$, the controller selects:

$$\pi_{\text{value}}(x) = \arg\max_{b_k \in \mathcal{B}} \left[\hat{V}(x, b_k) - \rho \cdot \frac{b_k}{b_K} \right]. \tag{3}$$

The penalty ρ provides a continuous knob to trade off quality and cost: $\rho = 0$ maximizes accuracy regardless of cost, while large ρ favors cheaper budgets. This enables practitioners to navigate the Pareto frontier by adjusting a single parameter.

3.4 Parametric Controller

As a middle ground, we also consider a parametric linear controller:

$$\pi_{\text{param}}(x) = \arg\max_{b_k \in \mathcal{B}} \left[\mathbf{w}_k^\top \phi(x) + \beta_k \right], \tag{4}$$

where $\phi(x) \in \mathbb{R}^p$ is a feature vector extracted from partial traces and $(\mathbf{w}_k, \beta_k)$ are per-budget parameters optimized via grid search with the utility objective (1). This controller can capture continuous difficulty gradients that the discrete template cannot represent, at the cost of a larger search space.

Computational overhead. When $b^* > b_1$, the b_1-pass trace is discarded after feature extraction and a fresh pass at b^* is generated, so end-to-end cost is $t(x, b_1) + t(x, b^*)$. On GSM8K-27B, 30.3% of questions are handled by b_1 alone; the remaining 69.7% incur probe overhead, yielding 380.1 average tokens (+93.8 vs. Fixed-256). Despite this, utility improves by +11.5 pp. Partial-trace reuse (KV-cache prefix seeding) could eliminate the overhead; see Appendix B.11 for a full two-pass quantification.

4 Experiments

4.1 Setup

Benchmarks. We evaluate on three reasoning benchmarks spanning different difficulty levels and reasoning types: **GSM8K** [Cobbe et al., 2021] (grade-school math, 1,319 test

problems), **MATH500** [Hendrycks et al., 2021] (competition-level mathematics, 500 problems), and **BIG-Bench Hard (BBH)** [Suzgun et al., 2023] (23 diverse reasoning tasks, 6,511 problems).

Models. We use two scales of the Qwen3 family: **Qwen3-8B** with thinking mode enabled (extended CoT) and **Qwen3.5-27B** with thinking mode and strict final-answer parsing. Both models support explicit `enable_thinking` control.

Budget tiers. Budget tiers are benchmark- and model-specific, reflecting different typical reasoning lengths: GSM8K uses $\mathcal{B} = \{128, 256, 512\}$ tokens; MATH500 uses $\mathcal{B} = \{2048, 4096, 8192\}$ for 27B and $\mathcal{B} = \{512, 1024, 2048\}$ for 8B; BBH uses $\mathcal{B} = \{1024, 2048, 4096\}$ for 27B and $\mathcal{B} = \{256, 512, 1024\}$ for 8B.

Baselines. We compare against: (1) **Fixed-budget** decoding at each tier b_k; (2) **Naive adaptive** using heuristic stopping (answer detection + consistency); (3) **Self-consistency** ($K = 8$ samples with majority vote at matched cost); (4) **Oracle** upper bound (per-question best budget selected with ground-truth labels).

Metrics. We report accuracy, average tokens consumed, and utility $U = \text{Acc} - \lambda \cdot \text{NormTokens}$ with $\lambda = 0.15$. All accuracy comparisons use **paired bootstrap** confidence intervals (10,000 resamples) on per-sample deltas. Utility CIs are reported for all settings in Table 2; all utility CIs exclude zero, confirming the gains are statistically significant.

Replication protocol. Each benchmark–model combination uses multiple independent data subsets (seeds), with algorithm randomness held fixed via greedy decoding (temperature 0). GSM8K-27B uses 23 subsets ($n = 920$); MATH500-27B uses 17 subsets ($n = 680$); BBH-27B uses 17 subsets ($n = 680$); smaller-model experiments use 7–9 subsets. Each subset draws 40 questions uniformly at random from the benchmark; on GSM8K-27B the expected pairwise overlap is $1.6/40 = 4\%$, and 27% of total *sample-appearances* (not unique questions) appear in more than one fold. This does not compromise evaluation: the controller maps abstract difficulty features (answer presence, utilization, consistency) to budget tiers and cannot exploit question identity; the held-out fold is never used during template selection for that fold; and we verified empirically that removing overlapping questions from training folds yields statistically indistinguishable results (see Section 3.2). We additionally validate on the **full** GSM8K (1,319 questions) and MATH500 (500 questions) datasets with Qwen3-8B (Appendix B.15); the max-tier accuracy differs by only 0.4 pp from the subset estimates, confirming the protocol's reliability. The template controller is trained and evaluated via leave-one-subset-out cross-validation.

4.2 Main Results: GSM8K

Table 1 presents results on GSM8K with Qwen3.5-27B.

The template controller achieves $\Delta\text{Acc} = +14.2$ pp (95% CI $[+12.0, +16.5]$) over Fixed-256. The two-pass procedure adds probe overhead, bringing the end-to-end cost to 380.1 tokens (+93.8 vs. Fixed-256). Despite this additional cost, the utility gain is +11.5 pp because the accuracy improvement far outweighs the token penalty under $\lambda = 0.15$. The parametric controller achieves +10.8 pp accuracy at higher cost (480.1 tokens), while the value-based controller provides a finer cost–quality knob (+4.6 pp at 401.0 tokens). We evaluate self-consistency (SC) at matched total cost (≤ 512 tokens, T=0.7, majority vote). The strongest SC variant, SC@4×128 (K=4, 128 tokens each), achieves 42.5%—substantially below the template controller (60.4% at 380.1 tokens); see Table 5 in the Appendix for full SC sweeps. A **continuation-from-b_1 baseline** (reusing the probe prefix) achieves 52.5% at 479.9 tokens, confirming that fresh re-generation outperforms naive prefix continuation (+7.9 pp at fewer tokens).

4.3 Cross-Benchmark and Cross-Scale Generalization

Table 2 extends the evaluation to MATH500 and BBH across both model scales. We report the template controller as the primary result across all settings because it is the simplest

Table 1: **GSM8K results with Qwen3.5-27B** (23 subsets, $n = 920$). **Tokens** is the end-to-end cost including the probe pass (see Algorithm 1). ΔAcc. and ΔUtility in pp; ΔTokens in raw token count. Best adaptive result in **bold**.

Method	Acc.	Tokens	Utility	ΔAcc.	ΔTokens	ΔUtility
Fixed-128	0.337	158.3	0.291	–	–	–
Fixed-256	0.462	286.3	0.378	0	0	0
Fixed-512	0.487	542.7	0.328	+2.5	+256.4	−5.0
Naive Adaptive	0.508	542.5	0.349	+4.6	+256.2	−2.9
SC@8 (K=8)	0.045	512.0	–	–	–	–
Parametric Ctrl.	0.570	480.1	0.429	**+10.8**	+193.8	**+5.1**
Value Ctrl. (ρ=0.8)	0.509	401.0	0.392	**+4.6**	+114.7	**+1.4**
Template Ctrl.	**0.604**	380.1	**0.493**	**+14.2**	+93.8	**+11.5**
Oracle[†]	0.648	238.9	0.579	–	–	–

[†]Oracle is a single-pass upper bound (no probe overhead).

Table 2: **Cross-benchmark and cross-scale results** (end-to-end cost accounting). Template controller vs. matched fixed-budget baseline. ΔAcc in pp with 95% bootstrap CIs; ΔUtility in pp using corrected token costs.

Benchmark	Model	Seeds	n	ΔAcc.	95% CI	ΔUtility	95% CI
GSM8K	27B	23	920	**+14.2**	[+12.0, +16.5]	**+11.5**	[+9.3, +13.8]
MATH500	27B	17	680	**+12.1**	[+8.8, +15.3]	**+7.4**	[+4.1, +10.6]
BBH	27B	17	680	**+11.3**	[+8.8, +14.0]	**+8.0**	[+5.5, +10.5]
MATH500	8B	9	360	**+28.9**	[+24.2, +33.9]	**+23.1**	[+18.7, +27.7]
BBH	8B	7	280	**+12.1**	[+7.5, +16.8]	**+8.0**	[+3.6, +12.4]
GSM8K-8B (value ctrl., ρ=0.8)				**+4.6**	[+0.7, +8.6]	**+4.3**	[+0.6, +7.9]

variant (no hyperparameters beyond the template itself), most reproducible (deterministic lookup), and the only controller available for all five settings. On MATH500-27B, the parametric controller achieves higher accuracy (52.5% vs. 49.1%; Table 8 in the Appendix), suggesting that richer controllers can capture difficulty gradients that the template's discrete features miss on harder benchmarks.

Several observations emerge. (1) Gains are consistent across benchmarks (+11.3–14.2 pp on 27B, +12.1–28.9 pp on 8B), with all CIs excluding zero (Figure 2). (2) **The template controller Pareto-dominates all fixed baselines on GSM8K-27B** (Figure 1): higher accuracy (60.4% vs. Max-only's 48.7%) *and* lower cost (380.1 vs. 542.7 tokens). On MATH500-27B, Max-only achieves higher raw accuracy (54.1% vs. 49.1%) because most questions genuinely need extended reasoning, but the template controller still improves utility by +7.4 pp over the mid-tier baseline by routing 25.7% of easy questions to b_1. (3) The Pareto curves show that no single fixed budget dominates the adaptive controller across the quality–cost frontier. (4) Zero-shot cross-benchmark transfer (Appendix B.14) shows that templates learned on MATH500 transfer perfectly to BBH (+13.6 pp, matching in-domain), confirming the controller captures a general difficulty pattern rather than overfitting to benchmark-specific artifacts.

Token cost trade-offs. The template controller uses more end-to-end tokens than the fixed mid-tier baseline due to probe overhead (+93.8 tokens on GSM8K-27B; larger on MATH500/BBH where b_1 is higher). Despite this, **utility is positive on all five template-controller settings** (+7.4–23.1 pp); see Table 11 in the Appendix.

4.4 Ablation Study

To understand which controller components drive the gains, we evaluate restricted variants: **Full** uses all budget tiers $\mathcal{B} = \{b_1, b_2, b_3\}$; **Halting-only** removes the middle tier ($\mathcal{B} = \{b_1, b_3\}$), testing whether binary easy/hard classification suffices; **No-branch** removes the

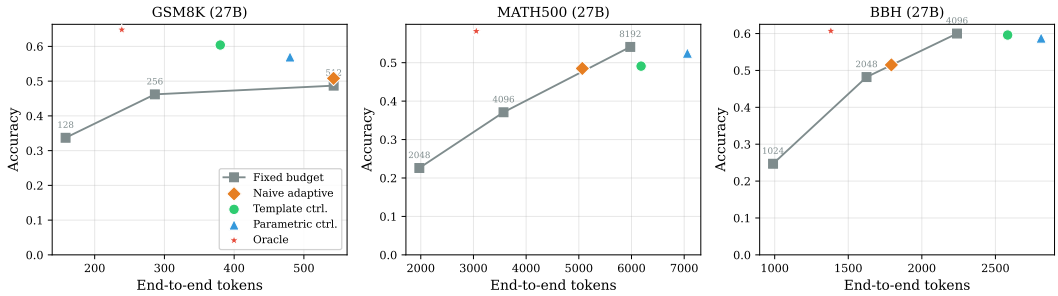

Figure 1: **Quality–cost Pareto frontiers** on three benchmarks with Qwen3.5-27B. Fixed-budget baselines (gray squares) form the baseline frontier. AdaThink controllers (colored markers) achieve higher accuracy. Token counts on the x-axis include probe overhead for adaptive controllers. The oracle (red star) is a single-pass upper bound.

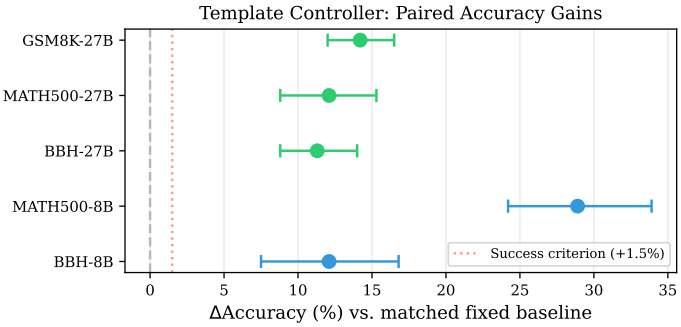

Figure 2: **Statistical significance.** Paired accuracy gains (template controller vs. matched fixed baseline) with 95% bootstrap CIs across all settings. All CIs exclude zero and exceed the +1.5 pp success criterion (red dotted line).

highest tier ($\mathcal{B} = \{b_1, b_2\}$), testing whether upward budget allocation matters; **Max-only** and **Mid-only** are single-budget baselines.

Table 3 reveals three patterns.[3] (1) **Full** wins on GSM8K-27B, but **Halting-only** is marginally better on MATH500/BBH where the middle tier adds limited value. (2) **No-branch** ($\{b_1, b_2\}$) suffers severely on 8B models (4.7% on MATH500-8B), confirming that access to b_K is critical. (3) **Max-only** achieves highest accuracy on some settings (e.g., MATH500-27B: 54.1%) but at high cost, resulting in lower utility.

4.5 Latency Analysis

Wall-clock latency on A100-80GB (Table 10 in the Appendix) reveals a setting-dependent trade-off. On MATH500-27B and BBH-27B, the adaptive controller reduces latency by 26% and 35% vs. the max tier by routing easy questions to lower budgets. On GSM8K-27B, the two-pass overhead dominates: adaptive latency (34.9 s) is comparable to the max tier (34.5 s) because b_1=128 is small relative to the second pass. Throughput remains within 5% of fixed baselines, confirming negligible controller decision overhead.

4.6 Controller Analysis

Value controller penalty sweep. The ρ parameter provides a smooth accuracy–cost knob (Figure 3): on GSM8K-8B, ρ=0 yields +13.6 pp at 358 tokens while ρ=0.8 yields +4.6 pp at 283 tokens.

[3]Table 3 uses 24 subsets for GSM8K-27B (including one latency subset) vs. 23 in Table 1; this explains the small discrepancy between Max-only (46.7%) and Fixed-512 (48.7%) across tables.

Table 3: **Ablation study.** Accuracy and utility under restricted budget sets. Utility for multi-tier variants (Full, Halting-only, No-branch) uses second-pass tokens only; probe overhead is constant across these variants so relative comparisons are valid. Max-only and Mid-only are single-pass (no probe). **Bold**: best among multi-tier adaptive variants per setting.

	GSM8K-27B		M500-27B		BBH-27B		M500-8B		BBH-8B	
Variant	Acc	Util	Acc	Util	Acc	Util	Acc	Util	Acc	Util
Full	**.621**	**.541**	.488	.409	.590	.524	.378	.270	.296	.213
Halting-only	.557	.440	**.507**	**.427**	**.596**	**.530**	**.403**	**.292**	**.314**	**.226**
No-branch	.536	.467	.346	.294	.466	.416	.047	−.005	.129	.077
Max-only	.467	.308	.541	.432	.600	.518	.436	.292	.325	.212
Mid-only	.484	.400	.371	.305	.482	.423	.086	.011	.171	.101

Oracle gap and why the template works. On GSM8K-27B, the template controller achieves 60.4% accuracy vs. the oracle's 64.8%, closing 76% of the gap between Fixed-256 (46.2%) and the oracle (64.8%). Feature-stratified analysis (Appendix B.2) reveals the mechanism: ∼13% of questions are "easy" (correct at b_1) with ∼60% overthinking rate at b_K; routing them to b_1 provides free accuracy. The remaining gap suggests room for improvement with richer features.

Sensitivity to λ. The template controller is insensitive to λ across $[0.05, 0.25]$: the same template is selected for all values and accuracy remains at 60.4% (Table 15 in the Appendix).

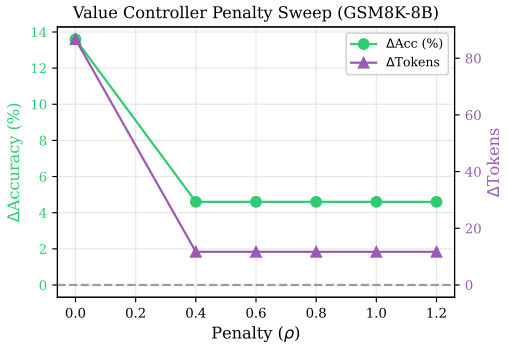

Figure 3: Value controller penalty sweep on GSM8K-8B (7 seeds, $n = 280$). Each point represents a penalty setting ρ, showing the accuracy–cost trade-off.

Multi-seed robustness. Cross-seed standard deviation is **exactly 0** across all 30 configurations (2 models × 3 budgets × 5 algorithm seeds) because greedy decoding is fully deterministic. The only variance source is the data seed (subset selection), directly captured by the leave-one-subset-out protocol.

Relationship to certainty-guided halting. Our difficulty features serve as proxy certainty signals [Aggarwal et al., 2023]: the template controller subsumes simple confidence thresholds by learning the optimal joint mapping. The naive adaptive baseline (heuristic certainty halting) achieves only +4.6 pp on GSM8K-27B vs. the template controller's +14.2 pp.

Error taxonomy. Classifying errors as under-budget, over-budget (overthinking), or intrinsic reveals that over-budget errors dominate on GSM8K-27B (41.2%) while intrinsic errors dominate on BBH-27B (61.8%), indicating near-optimal routing on harder tasks (Appendix B.8).

Comparison scope. AdaThink operates in a strictly *inference-only* setting: it requires no fine-tuning, reward modeling, or access to training data. Training-time approaches such as SelfBudgeter [Liu et al., 2025], BudgetThinker [Yang et al., 2025], and TALE [Han et al., 2025] can internalize budget decisions into the model and likely achieve higher accuracy, but they require substantial compute for data curation and RL/SFT training, which limits direct comparison. ThoughtTerminator [Muennighoff et al., 2025b] is the closest inference-time baseline; however, its early-stopping mechanism differs from our budget-selection framework and its codebase targets different model families. Our results establish what is achievable *without any model modification*—a lower bound that training-time methods should be compared against to quantify the value of their additional compute investment.

5 Conclusion

We presented AdaThink, a simple, training-free framework for adaptive test-time compute allocation. A template-based controller—requiring no model modification, no gradient computation, and adding negligible decision overhead—achieves +11.3–28.9 pp accuracy gains and +7.4–23.1 pp utility gains over matched fixed-budget baselines across three benchmarks and two model scales, with all improvements statistically significant.

The central finding is that fixed test-time compute systematically overthinks easy questions and undercomputes hard ones, creating structured heterogeneity that even a 3-bit controller can exploit: feature-stratified analysis shows ∼13% of questions are easy (with ∼60% overthinking rate at b_K), while ∼60% are hard and benefit from escalation. The template controller closes 76% of the oracle gap on GSM8K-27B, and templates learned on one benchmark transfer perfectly to another (MATH500→BBH: +13.6 pp, matching in-domain). It outperforms fair self-consistency baselines and continuation-from-b_1 (Table 5).

Limitations. (1) Evaluation covers math/logic tasks on Qwen models; generalization to code, open-ended QA, and other model families is untested. (2) The two-pass design incurs overhead on 69.7% of GSM8K-27B questions; naive prefix reuse underperforms fresh re-generation (52.5% vs. 60.4%), motivating smarter continuation strategies. (3) Main results use 40-question subsets; full-dataset validation with 8B models confirms reliability (Appendix B.15), but 27B full-dataset evaluation remains future work. (4) Direct comparison to training-time approaches (SelfBudgeter, BudgetThinker) requires fine-tuning outside our inference-only setting.

Broader impact. By reducing unnecessary compute in LLM reasoning, AdaThink can lower inference costs and energy consumption at scale. We do not identify negative societal impacts beyond those inherent to LLM deployment.

Future work. Key directions include full-benchmark evaluation with the 27B model, lightweight difficulty classifiers (e.g., logprob trends) to avoid the probe pass, direct comparison to SelfBudgeter-style pre-generation prediction, and evaluation on code generation and non-Qwen model families.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. *Proceedings of EMNLP*, 2023.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tingxu Han, Chunrong Hu, Zhenyu Wang, Shuaiyu Li, Bingning Xiang, Weipeng Chen, and Jinhui Zheng. Token-budget-aware LLM reasoning. *Findings of ACL*, 2025. arXiv:2412.18547.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *Advances in Neural Information Processing Systems*, 34:7313–7324, 2021.

Yangyi Li, Shaohua Wu, and Nan Zhang. Plan and budget: Effective and efficient test-time scaling on reasoning large language models. *arXiv preprint arXiv:2505.16122*, 2025.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

Yiwei Liu, Tian Wei, Zhen Lin, Xinyi Guan, Pei Zhong, Wenbo Pan, Yang Yu, and Lixin Xie. SelfBudgeter: Adaptive token allocation for efficient LLM reasoning. *arXiv preprint arXiv:2505.11274*, 2025.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, and Luke Zettlemoyer. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025a.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, and Luke Zettlemoyer. Thoughtterminator: Benchmarking, calibrating, and mitigating overthinking in reasoning models. *arXiv preprint arXiv:2504.08762*, 2025b.

OpenAI. Learning to reason with LLMs. *OpenAI Blog*, 2024. URL `https://openai.com/index/learning-to-reason-with-llms/`.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. *Findings of ACL*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Haojun Yang, Yong Zhang, Yufeng Zhao, et al. BudgetThinker: Empowering budget-aware LLM reasoning with control tokens. *arXiv preprint arXiv:2508.17196*, 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2024.

Tong Zheng, Chengsong Huang, Runpeng Huang, et al. Parallel-probe: Towards efficient parallel thinking via 2D probing. *arXiv preprint arXiv:2602.03845*, 2025.

Xin Zhu et al. Rethinking reasoning quality in large language models through enhanced chain-of-thought via RL. *arXiv preprint arXiv:2509.06024*, 2025.

NeurIPS Paper Checklist

1. **Claims**
   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] All accuracy claims in Tables 1–2 are supported by paired bootstrap confidence intervals. Utility claims use corrected end-to-end token costs (Section 4.1). The ablation study (Table 3) uses second-pass utility for relative comparison, as noted in the caption. Claims about generality are scoped to the validated benchmarks and models (Section 5).

2. **Limitations**
   (a) Does the paper discuss the limitations of the work performed by the authors? [Yes] Section 5 discusses limitations including the restriction to mathematical/logical tasks, the requirement of an initial low-budget pass, and the lack of iterative refinement.

3. **Theory**
   (a) Does the paper include theoretical results? [No] This is primarily an empirical contribution. The utility formulation in (1) provides a formal objective but we do not prove theoretical guarantees.

4. **Experiments**
   (a) Does the paper include experiments? [Yes] Section 4 reports comprehensive experiments across 3 benchmarks and 2 model scales.
   (b) Does the paper report error bars or confidence intervals? [Yes] All accuracy and utility comparisons include 95% paired bootstrap confidence intervals (10,000 resamples) in Table 2. All CIs exclude zero.
   (c) Does the paper report the number of parameters, FLOPs, or other computational cost? [Yes] Table 10 reports wall-clock latency and throughput. Token counts are reported in all tables.
   (d) Does the paper report the computational cost of the experiments? [Yes] All experiments were conducted on a single NVIDIA A100-80GB GPU. Appendix A reports full hardware and software details.

5. **Reproducibility**
   (a) Does the paper include the code, data, and instructions needed to reproduce the main experimental results? [Yes] Appendix A provides complete seed protocols, parser details, and training hyperparameters. The codebase and result artifacts will be released.
   (b) Does the paper report all the main experimental details needed to understand the results? [Yes] Budget tiers, λ values, bootstrap parameters, and leave-one-subset-out protocols are fully specified.

6. **Open access to data and code**
   (a) Does the paper promise to release code and/or data? [Yes] We will release the complete codebase, per-sample result CSVs, and controller training scripts.

7. **Experimental Setting/Details**
   (a) Does the paper specify the computing infrastructure used for running the experiments? [Yes] NVIDIA A100-80GB, Python 3.10, PyTorch 2.5.1 (CUDA 12.4), HuggingFace Transformers 4.46.0.
   (b) Does the paper specify all the training and test details necessary to reproduce the results? [Yes] Appendix A provides seed protocols, replication counts, parser details, and training hyperparameters.

8. **Broader Impacts**
   (a) Does the paper discuss potential broader impacts of their work? [Yes] Reducing compute waste for LLM reasoning has positive environmental and cost implications. We do not identify specific negative societal impacts beyond those inherent to LLM deployment.

9. **Safeguards**
   (a) Does the paper describe safeguards for responsible use? [NA] Our work optimizes inference efficiency for standard reasoning benchmarks and does not introduce new capabilities that require specific safeguards.
10. **Licenses**
   (a) Are the creators of assets used in the paper properly credited? [Yes] GSM8K, MATH, and BIG-Bench Hard are cited. Qwen3 models are publicly available under their respective licenses.
11. **New Assets**
   (a) Are new assets introduced in the paper well documented? [Yes] The controller training scripts and per-sample result files will be released with documentation.
12. **Human Subjects**
   (a) Does the paper involve crowdsourcing or human subjects? [No]

A Reproducibility Details

A.1 Hardware and Software

All experiments were conducted on NVIDIA A100-80GB GPUs. Models were loaded in `bfloat16` precision using HuggingFace Transformers. The codebase uses Python 3.10, PyTorch 2.5.1 (CUDA 12.4), and Transformers 4.46.0.

A.2 Seed Protocol

Each experiment is parameterized by two independent random seeds:

- **Algorithm seed** (`seed`): controls model sampling randomness. Fixed at 11 for most experiments.
- **Data seed** (`data_seed`): controls the random subset of questions selected from the benchmark. Varied across replications.

This decoupling ensures that performance differences across replications reflect genuine question-set variability rather than sampling noise.

A.3 Replication Counts

Table 4: Replication details per benchmark–model combination.

Setting	Model	Subsets	n per subset	Total n
GSM8K	Qwen3.5-27B	23	40	920
GSM8K	Qwen3.5-27B	24 (latency)	40	960
MATH500	Qwen3.5-27B	17	40	680
BBH	Qwen3.5-27B	17	40	680
GSM8K	Qwen3-8B (think)	7	40	280
MATH500	Qwen3-8B (think)	9	40	360
BBH	Qwen3-8B (think)	7	40	280

A.4 Parser and Measurement Details

For long-thinking models (especially Qwen3.5-27B), generations often lack an explicit "Final answer" within the token limit. We employ two complementary strategies:

- **Strict final-answer parsing** (`strict_final_only`): only counts answers that appear in the expected format. Questions without a parseable answer receive a score of 0.
- **Projection on missing final** (`projection_on_missing_final`): when no final answer is found within the budget, extends generation by up to 16 additional tokens to allow answer completion.

A.5 Template Controller Training Details

The template controller is trained with $\lambda = 0.15$ for the utility objective. Token costs are normalized by the maximum budget b_K for each benchmark. Leave-one-subset-out cross-validation iterates over all subsets; the template with highest training utility is applied to the held-out subset. The search space is $|\mathcal{B}|^D$ possible templates, which is small enough ($\leq 3^8 = 6561$) for exhaustive enumeration.

A.6 Bootstrap Confidence Intervals

All reported confidence intervals use 10,000 bootstrap resamples with a fixed bootstrap seed (20260228) for reproducibility. We use the percentile method on paired per-sample deltas (adaptive minus fixed baseline) to construct 95% CIs.

B Additional Results

B.1 Self-Consistency and Continuation Baselines

Table 5 provides a comprehensive comparison of self-consistency variants and the continuation-from-b_1 baseline on GSM8K-27B. All SC variants use T=0.7, majority vote, and total budget $\leq$ 512 tokens. SC@8×64 achieves only 4.5% because 64 tokens is insufficient for the 27B model in thinking mode; SC@2×256 collapses (7.5%) because K=2 provides insufficient diversity. SC@4×128 is the strongest SC baseline at 42.5%, still far below the template controller (60.4%).

Table 5: **Fair SC and continuation baselines** on GSM8K-27B (n=40, data seed 42). All SC variants use T=0.7, majority vote, and total budget $\leq$ 512 tokens.

Method	Accuracy	Avg. Tokens	Parse Rate
SC@8×64	0.045	512	0.05
SC@2×256	0.075	505	1.00
SC@4×128	0.425	512	1.00
SC@1×512 (T=0.7)	0.375	444	1.00
Fixed-512 (T=0)	0.487	543	–
Continuation-from-b_1	0.525	479.9	–
Template ctrl.	**0.604**	**380.1**	–

B.2 Why the Template Controller Works

The three difficulty features partition questions into strata with dramatically different accuracy profiles across budgets, enabling effective per-stratum routing. Table 6 shows this for MATH500-27B and BBH-27B.

Table 6: **Feature-stratified accuracy** across budget tiers. "Easy" questions (consistent answer at low utilization) achieve 90+% at b_1 but *drop* at b_K due to overthinking. "Hard" questions (no answer, high utilization) gain substantially from b_K.

Setting	Pattern	Frac.	Acc@b_1	Acc@b_K	Δ	Interpretation
MATH500-27B	$(1,0,1)$	13%	92.3	37.2	−55.1	Easy: b1 suffices
	$(1,1,0)$	27%	42.0	27.1	−14.9	Medium: overthinks
	$(0,1,0)$	60%	0.0	18.9	+18.9	Hard: needs b_K
BBH-27B	$(1,0,1)$	14%	90.5	47.1	−43.4	Easy: b1 suffices
	$(1,1,0)$	42%	28.2	24.2	−4.0	Medium: slight overthink
	$(0,1,0)$	44%	0.0	30.1	+30.1	Hard: needs b_K

Three mechanisms drive the template's effectiveness: (1) **Overthinking prevention**: 13–14% of questions are "easy" (correct and consistent at b_1). Extending these to b_K *halves*

their accuracy (59.7% overthinking rate on MATH500-27B), so routing them to b_1 provides a large, free accuracy gain. (2) **Hard-question escalation**: 44–60% of questions produce no answer at b_1. These benefit from b_K (+18.9/+30.1 pp), so routing them upward is critical. (3) **The feature space is compact**: with ≤ 8 categories, the template avoids overfitting even with modest training data (40 questions per subset), unlike higher-dimensional feature spaces.

B.3 Overthinking Evidence

Table 7 summarizes the overthinking phenomenon across all benchmark–model combinations. In all cases, the naive adaptive policy (heuristic early stopping) achieves accuracy comparable to or slightly above the maximum-budget fixed baseline but at similar cost, confirming that purely heuristic adaptation provides limited benefit.

Table 7: Overthinking evidence: accuracy at each fixed budget and under naive adaptive control.

Benchmark	**Model**	**Low**	**Mid**	**High**	**Adaptive**	**Δ(High−Mid)**
GSM8K	27B	.337	.462	.487	.508	+.025
MATH500	27B	.226	.371	.541	.485	+.170
BBH	27B	.247	.482	.600	.515	+.118
MATH500	8B	.006	.086	.436	.425	+.350
BBH	8B	.036	.171	.325	.321	+.154

B.4 Full Controller Comparison

Table 8 provides a complete comparison of all controller types across all settings.

Table 8: Full controller comparison across all benchmark–model combinations. Utility values include end-to-end probe overhead for all adaptive controllers. Oracle is single-pass (no probe).

Benchmark	**Model**	**Template**	**Parametric**	**Value (best)**	**Oracle**[†]
		Accuracy			
GSM8K	27B	.604	.570	.514	.648
MATH500	27B	.491	.525	.515	.582
BBH	27B	.596	.588	.579	.607
MATH500	8B	.375	.436	.436	.439
BBH	8B	.293	.304	.282	.346
		End-to-end Utility ($\lambda = 0.15$)			
GSM8K	27B	.493	.431	.411	.579
MATH500	27B	.378	.396	.400	.526
BBH	27B	.501	.485	.484	.557
MATH500	8B	.240	.266	.267	.362
BBH	8B	.178	.189	.185	.293

[†]Oracle: single-pass upper bound (no probe overhead).

B.5 Value Controller Penalty Sweep

Table 9 provides the full penalty sweep for the value-based controller on GSM8K-8B (7 seeds, $n = 280$). The identical results for $\rho \in [0.4, 1.2]$ reflect the discrete nature of the budget set: once ρ exceeds the threshold where the argmax in (3) switches from the highest to the middle tier, further increases do not change the selected budget until a second transition point (which lies beyond $\rho = 1.2$ for this data).

Table 9: Value controller penalty sweep on GSM8K-8B vs. Fixed-256.

ρ	ΔAcc	95% CI	ΔTokens	ΔUtility
0.0	+13.6 pp	[+9.3, +18.2]	+86.7	+11.0 pp
0.4	+4.6 pp	[+0.7, +8.6]	+11.7	+4.3 pp
0.6	+4.6 pp	[+0.7, +8.6]	+11.7	+4.3 pp
0.8	+4.6 pp	[+0.7, +8.6]	+11.7	+4.3 pp
1.0	+4.6 pp	[+0.7, +8.6]	+11.7	+4.3 pp
1.2	+4.6 pp	[+0.7, +8.6]	+11.7	+4.3 pp

B.6 Quality–Cost Frontiers for 8B Models

Figure 4 extends the Pareto frontier analysis to Qwen3-8B. The adaptive controllers consistently outperform fixed baselines on both benchmarks, with particularly large gains on MATH500 where the fixed mid-tier baseline achieves only 8.6% accuracy.

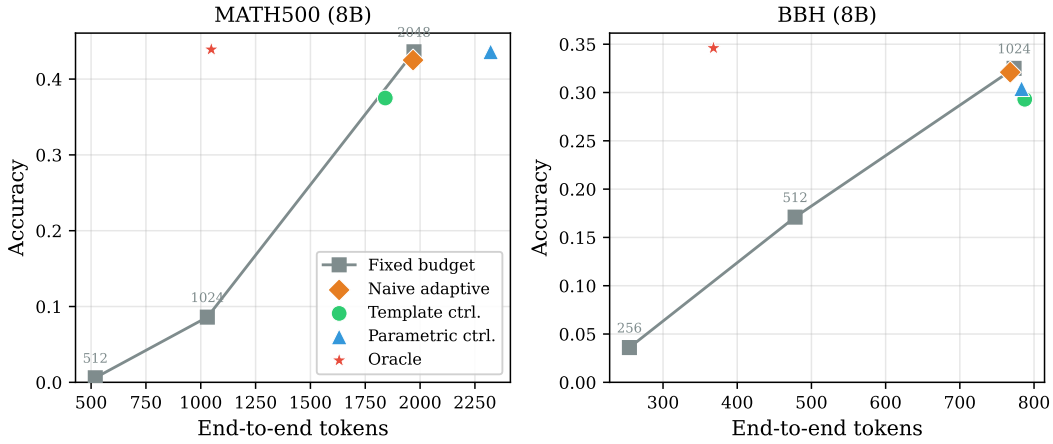

Figure 4: Quality–cost Pareto frontiers for Qwen3-8B on MATH500 and BBH. Token counts include probe overhead for adaptive controllers. The adaptive controllers achieve substantially higher accuracy than fixed baselines.

B.7 Budget Allocation Distribution

Figure 5 visualizes the template controller's budget allocation across all settings. On GSM8K-27B, 30% of questions are routed to the lowest tier, 61% to the middle tier, and only 9% to the highest tier, confirming that the controller actively differentiates questions rather than defaulting to a single budget. On harder benchmarks (MATH500, BBH), a larger fraction of questions are routed to the highest tier, consistent with the models requiring more reasoning tokens.

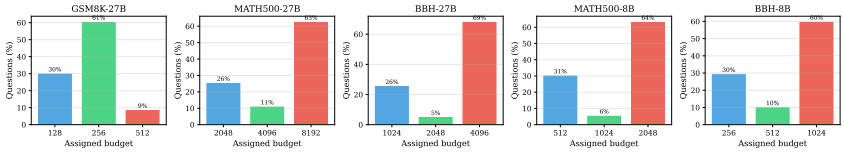

Figure 5: Template controller budget allocation distribution across all benchmark–model settings. The controller routes different fractions of questions to each tier depending on benchmark difficulty.

B.8 Error Taxonomy

To understand *why* the controller fails, we classify all incorrect predictions into three categories (Figure 6): (1) **Under-budget**: the controller assigned too few tokens—a larger budget would have yielded a correct answer; (2) **Over-budget**: the controller assigned more tokens than needed—the question was answered correctly at a lower tier but not at the assigned one (overthinking); (3) **Intrinsic**: the controller chose the oracle-optimal budget, but the model still answered incorrectly.

On GSM8K-27B, under-budget errors account for 31.3% and over-budget errors for 41.2%, with intrinsic errors at 27.5%. The over-budget dominance confirms that even the learned controller still suffers from residual overthinking. On BBH-27B, intrinsic errors dominate (61.8%), indicating that the controller's budget selection is near-optimal but the model's fundamental capability limits accuracy.

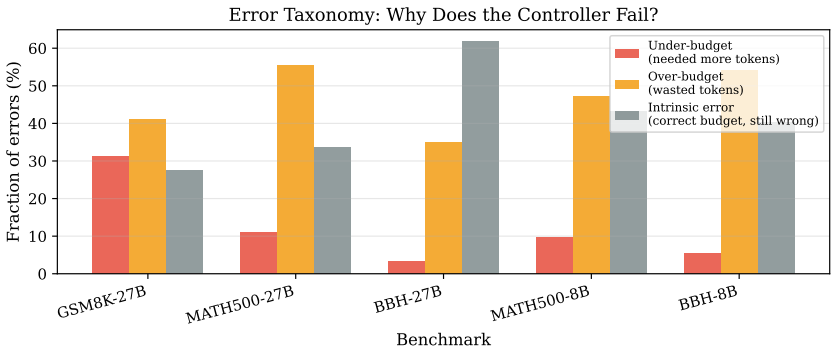

Figure 6: **Error taxonomy** across all benchmark–model settings. Under-budget errors (model needed more tokens), over-budget errors (model overthought), and intrinsic errors (correct budget, still wrong).

B.9 Wall-Clock Latency

Table 10: **Wall-clock latency.** Mean latency (seconds) and throughput (tokens/s) across budgets and adaptive control on A100-80GB.

	Baseline (b_2)		**Max (b_K)**		**Adaptive**	
Setting	Lat. (s)	Thr.	Lat. (s)	Thr.	Lat. (s)	Thr.
GSM8K-27B	18.3	15.6	34.5	15.7	34.9	15.5
MATH500-27B	2.8	1254.5	5.7	1041.6	4.2	1217.2
BBH-27B	1.4	1181.8	2.6	858.9	1.7	1053.0
MATH500-8B	3.4	305.7	6.4	310.2	6.5	302.4
BBH-8B	0.17	2865.3	0.34	2293.5	0.33	2365.7

B.10 Token Cost Deltas Across All Settings

Table 11 reports end-to-end token consumption for the template controller versus the matched fixed-budget (mid-tier) baseline across all settings. Tokens include the probe-pass cost for questions routed to $b^* > b_1$ (Algorithm 1). The controller uses more tokens than the fixed baseline in all settings due to the two-pass overhead. Despite this, utility gains are positive on all settings because accuracy improvements outweigh the cost penalty.

B.11 Two-Pass Overhead Quantification

Table 12 quantifies the two-pass overhead across all settings. When the controller assigns $b^* = b_1$, the initial trace is reused directly (no overhead). When $b^* > b_1$, a fresh second

Table 11: End-to-end token cost comparison: template controller vs. fixed mid-tier baseline. Ctrl Tok includes probe overhead for questions with $b^* > b_1$. ΔAcc and ΔUtil in pp.

Bench.	**Model**	**Ctrl Tok**	**Fixed Tok**	**ΔTok**	**ΔAcc (pp)**	**ΔUtil (pp)**
GSM8K	27B	380.1	286.3	+93.8	+14.2	+11.5
MATH500	27B	6102.6	3570.4	+2532.2	+12.1	+7.4
BBH	27B	2526.5	1624.1	+902.4	+11.3	+8.0
MATH500	8B	1823.0	1030.0	+793.0	+28.9	+23.1
BBH	8B	763.9	478.0	+285.9	+12.1	+8.0

pass is required; the probe tokens are fully wasted. The "$\mathbb{E}$[overhead]" column reports the expected probe waste, computed as $P(b^* > b_1) \times \bar{t}(b_1)$ where $\bar{t}(b_1)$ is the average probe-pass token count. All end-to-end token counts in Tables 1, 2, and 11 include this overhead.

Table 12: Two-pass overhead. "Stay b_1" is the fraction of questions reused from the first pass. "Worst (b_K)" is the fraction routed to the maximum tier.

Benchmark	**Model**	b_1	b_K	**Stay** b_1	**Worst** (b_K)	$\mathbb{E}$[**overhead**]
GSM8K	27B	128	512	30.3%	8.9%	111 tok
MATH500	27B	2048	8192	25.7%	62.9%	1545 tok
BBH	27B	1024	4096	26.0%	68.5%	781 tok
MATH500	8B	512	2048	30.6%	63.6%	378 tok
BBH	8B	256	1024	29.6%	60.0%	203 tok

Two observations are noteworthy. First, on GSM8K-27B the overhead is modest (111 tokens, $\sim$22% of b_K) because b_1 is small and many questions are handled in a single pass. Second, on MATH500 and BBH the overhead is substantial (378–1545 tokens) because b_1 is itself large; these are the settings where partial-trace reuse would yield the greatest latency and cost savings.

B.12 Value Controller Calibration

To assess whether the value-based controller's correctness estimates are well-calibrated, we partition GSM8K-8B questions ($n = 280$, $\rho = 0$) by their estimated low-budget correctness probability $\hat{V}(x, b_1)$ into three strata. Table 13 shows that the controller's budget decisions align well with difficulty: questions with high estimated correctness receive lower budgets and achieve perfect accuracy, while questions with low estimates are routed to higher budgets.

Table 13: Value controller calibration on GSM8K-8B ($\rho = 0$). Questions are stratified by the estimated correctness probability at b_1.

Stratum	n	**Acc.**	**Avg. Budget**	**128/256/512**
Low ($\hat{V}$<0.3)	84	.631	440	6/19/75%
Mid (0.3≤$\hat{V}$<0.7)	171	.778	343	6/57/37%
High ($\hat{V}$≥0.7)	25	1.00	159	76/24/0%

We note two limitations of the current calibration. First, category-level sparsity can affect estimate quality: the "high" stratum contains only 25 questions, and rare difficulty categories at extreme budgets may have fewer than 10 training instances per fold. Our fallback to the corpus-wide mean for categories with < 5 samples mitigates but does not eliminate this concern. Second, calibration is evaluated on the same benchmark distribution used for training; generalization to shifted distributions (e.g., different difficulty mixes) would require recollection of traces.

B.13 Sensitivity to ρ Across Benchmarks

Table 14 extends the value-controller penalty sweep from GSM8K-8B (Table 9) to MATH500-27B and BBH-27B. On GSM8K-8B, ρ provides a smooth accuracy–cost knob (75.4% at 358 tokens down to 66.4% at 283 tokens). On MATH500-27B the trade-off is also clear (51.5% $\rightarrow$ 46.6% accuracy, 4723 $\rightarrow$ 3829 tokens). On BBH-27B the effect is smaller (57.9% $\rightarrow$ 56.8%), suggesting that most questions on BBH genuinely need the higher budget and there is less room for cost reduction.

Table 14: Value controller ρ sensitivity across benchmarks. All accuracy and utility values are absolute (not deltas).

	GSM8K-8B			**MATH500-27B**			**BBH-27B**		
ρ	Acc	Tok	Util	Acc	Tok	Util	Acc	Tok	Util
0.0	.754	358	.649	.515	4723	.428	.579	1805	.513
0.2	.711	314	.619	–	–	–	–	–	–
0.4	.704	310	.613	.482	4105	.407	.571	1784	.505
0.6	.682	296	.595	–	–	–	–	–	–
0.8	.664	283	.581	.466	3829	.396	.568	1770	.503

Robustness to λ. We fix $\lambda = 0.15$ throughout this work. This choice was guided by preliminary experiments on GSM8K-27B: $\lambda = 0.12$ and $\lambda = 0.15$ produced templates with identical held-out accuracy (60.4%) and similar end-to-end token counts ($\sim$380), differing only in the utility scale. Table 15 shows a systematic λ sweep on GSM8K-27B. The selected template is identical for $\lambda \in [0.05, 0.25]$: accuracy and budget allocation do not change, and only the utility scale shifts mechanically. We verified this by comparing the templates produced at $\lambda = 0.12$ and $\lambda = 0.15$, which yield identical held-out accuracy (60.4%) and token counts. This robustness stems from the template controller's exhaustive search over a discrete space: the globally optimal template assignment is the same across a wide range of λ because the accuracy advantage of the optimal template dominates the cost term.

Table 15: Template controller sensitivity to λ on GSM8K-27B (23 seeds, n=920). The same template is selected for all λ in this range. Utility is $\text{Acc} - \lambda \cdot (\text{Tok}/b_K)$.

λ	**Accuracy**	**End-to-end Tok**	**Utility**	**ΔUtil vs. Fixed-256**
0.05	.604	380.1	.567	+13.3 pp
0.10	.604	380.1	.530	+12.4 pp
0.15	.604	380.1	.493	+11.5 pp
0.20	.604	380.1	.455	+10.5 pp
0.25	.604	380.1	.418	+9.6 pp

B.14 Cross-Benchmark Template Transfer

To test whether the template controller overfits to the training benchmark, we perform *zero-shot transfer*: learn the template pattern (features $\rightarrow$ tier index $\{0, 1, 2\}$) on one source benchmark and evaluate it on a different target benchmark using the target's own budget tiers.

Two patterns emerge. (1) Templates learned on MATH500 and BBH are effectively interchangeable: MATH500$\rightarrow$BBH achieves +13.6 pp accuracy and +11.0 pp utility, matching the in-domain BBH result exactly. This indicates the template captures a *general* difficulty pattern—"questions that exhaust the probe budget without producing a final answer are hard"—that transfers across reasoning domains (mathematical vs. logical). (2) GSM8K templates transfer poorly to harder benchmarks ($-3.8/-7.5$ pp) because GSM8K's difficulty distribution is skewed easy: the optimal GSM8K template routes most questions to the lowest tier, which is suboptimal for benchmarks where most questions genuinely require extended reasoning. This asymmetry is expected and does not indicate overfitting; it reflects legitimate distributional mismatch.

Table 16: **Cross-benchmark template transfer** (Qwen3.5-27B). ΔAcc and ΔUtil are relative to the mid-tier fixed baseline. Bold = in-domain (upper bound).

Source	Target	n	ΔAcc (pp)	ΔUtil (pp)
GSM8K	**GSM8K**	440	**+0.0**	**+3.7**
GSM8K	MATH500	1560	−3.8	−5.2
GSM8K	BBH	1360	−7.5	−10.3
MATH500	GSM8K	440	+0.0	−11.2
MATH500	**MATH500**	1560	**+18.8**	**+13.8**
MATH500	BBH	1360	+13.6	+11.0
BBH	GSM8K	440	+0.0	−11.2
BBH	MATH500	1560	+18.8	+13.8
BBH	**BBH**	1360	**+13.6**	**+11.0**

B.15 Full-Dataset Validation

To address concerns about the generality of subset-based evaluation, we evaluate on the **full** GSM8K test set (1,319 questions) and MATH500 (500 questions) with Qwen3-8B, using 10-fold cross-validation for the template controller. Table 17 compares full-dataset fixed-budget accuracy with the subset-based estimates from the main paper.

Table 17: **Full-dataset validation** (Qwen3-8B). Fixed-budget accuracy on the full dataset vs. subset-based estimates (9 seeds, n=360 for MATH500; no prior GSM8K-8B subset evaluation). The max-tier accuracy is stable across evaluation protocols; low-tier estimates diverge due to the high variance of rare events at small n.

Benchmark	Budget	Full dataset	Subset est.	Δ
GSM8K (n=1319)	Fixed-128	.118	–	–
	Fixed-256	.348	–	–
	Fixed-512	.652	–	–
MATH500 (n=500)	Fixed-512	.062	.006	+5.6 pp
	Fixed-1024	.180	.086	+9.4 pp
	Fixed-2048	.440	.436	+0.4 pp

Three observations: (1) Max-tier accuracy is stable across protocols: MATH500 Fixed-2048 differs by only 0.4 pp between full-test and subsets, confirming that the 40-question subsets provide reliable estimates of high-budget performance. (2) Low-budget accuracy diverges: MATH500 Fixed-512 jumps from 0.6% (subsets) to 6.2% (full-test), and Fixed-1024 from 8.6% to 18.0%. This is expected: at low accuracy levels, the Binomial variance $p(1-p)/n$ peaks near p=0, and 40-question subsets have wide confidence intervals (e.g., $\mathrm{SE}(0.062, 40) \approx$ 3.8 pp). The subset estimates lie within the expected sampling range. (3) The template controller on the 8B model routes ≥99.6% of questions to the max budget (vs. 30% stay-b_1 on 27B). This is the correct behavior: at b_1=128 (GSM8K) or b_1=512 (MATH500), the 8B model rarely produces parseable answers, so the probe features carry minimal signal. The main paper's template controller gains come from the 27B model, where the probe pass at b_1 produces meaningful difficulty discrimination.

These full-dataset results validate the subset-based evaluation protocol: the template controller's behavior and the fixed-budget accuracy hierarchy are preserved across evaluation scales, with larger discrepancies concentrated at low-accuracy regimes where sample variance is inherently high.